

瑞芯微赛题——基于 RK3588 的端侧 AI 智能安防系统与多目标检测及实时推理优化

摘要

随着智慧城市、物联网和边缘计算技术的发展，传统安防系统普遍依赖云端服务器完成视频分析与数据存储，存在传输延迟高、网络带宽占用大、隐私泄露风险高以及离线场景适应能力不足等问题。针对上述需求，本作品设计并实现了一套基于 RK3588 的端侧 AI 智能安防系统。系统以瑞芯微 RK3588 嵌入式平台为核心，利用其内置 NPU 完成本地 AI 推理，在开发板端实现视频采集、目标检测、场景规则判断、报警响应与 Web 可视化展示的闭环处理，减少对云端服务器的依赖，提高系统实时性与数据安全性。

在算法与部署方面，系统采用 YOLO11n 轻量化目标检测模型作为基础感知网络，构建了 PyTorch→ONNX→RKNN 的模型转换与部署流程，并通过 RKNN-Toolkit2 将模型适配至 RK3588 NPU 平台运行。针对端侧算力和实时性要求，系统结合轻量化模型、NPU 推理和跳帧调度机制，在保证画面连续性的同时降低计算负载，提高整体运行稳定性。图像预处理部分采用 OpenCV 完成尺寸变换、颜色空间转换与输入张量构造，并预留 RGA 硬件加速优化空间，为后续进一步降低 CPU 占用提供扩展基础。

在系统功能方面，本作品采用“统一检测模型 + 多场景规则引擎”的解耦架构。系统基于同一 YOLO11n 检测结果，通过不同业务规则实现控制中心综合监测、商场客流统计、禁区 ROI 入侵检测、校园人员聚集预警、工厂危险区域管控、停车场车辆辅助监测、仓库异常进入预警以及家庭白名单识别等多种应用模式。各模式通过 Flask Web 控制台统一管理，后端以 MJPEG 视频流实时推送监控画面，并通过状态接口输出人数统计、报警等级、FPS、推理时延和事件记录等关键指标，前端支持模式切换、状态显示和报警可视化交互。

在家庭安防场景中，系统进一步设计了轻量级白名单机制。该机制基于 OpenCV Haar Cascade 完成人脸检测，并结合灰度归一化与模板比对实现端侧身份筛查。在不部署复杂人脸识别大模型的前提下，系统能够实现白名单人员正常

通行、陌生人员触发报警的功能，体现了资源受限嵌入式环境下的实用性和工程可落地性。

经测试，系统能够在 RK3588 平台上稳定完成实时目标检测与多场景报警展示，检测帧率可达到 25 FPS 以上，单帧推理时延控制在 40 ms 以内，具备较好的实时性、交互性和场景扩展能力。该系统可应用于智慧园区、校园安防、仓储物流、工业危险区域管控、家庭看护及轻量级边缘智能监控等场景，具有一定的工程应用价值和推广潜力。

第一部分 作品概述

1.1 功能与特性

本系统定位为面向边缘端部署的 AI 智能安防平台，以“单模型多场景复用、端侧闭环推理、Web 可视化交互”为核心设计理念，构建了从视频采集、目标检测、规则判断、报警响应到前端展示的完整功能链路。系统不依赖云端服务器即可完成实时监控与智能报警，适用于对实时性、隐私性和部署灵活性要求较高的边缘安防场景。

在视频感知层，系统通过 V4L2 接口驱动 USB 摄像头采集 640×480 实时视频流，并对图像进行尺寸调整、颜色空间转换和输入张量构造等预处理操作。预处理后的图像送入 YOLO11n 轻量化目标检测模型，由 RK3588 平台的 NPU 完成端侧推理，输出目标类别、置信度和边界框信息。为平衡推理负载与画面显示流畅度，系统采用跳帧推理机制，即部分帧执行 NPU 推理，部分帧复用上一帧检测结果，从而降低连续推理带来的计算压力，提升系统长时间运行的稳定性。

在业务决策层，系统采用“检测模型 + 场景规则引擎”的解耦设计。`detector.py` 负责统一完成目标检测与后处理，`camera.py` 根据当前场景模式对检测结果进行规则解释和报警判断。系统支持控制中心、商场、禁区、校园、工厂、停车场、仓库和家庭八种模式，分别面向综合监测、客流统计、ROI 区域入侵、人员聚集预警、危险区域管控、车辆辅助监测、仓库异常进入和家庭白名单识别等应用需求。不同模式共享同一检测模型，但采用不同规则进行结果判断，从而实现多场景复用与快速扩展。

在交互展示层，系统基于 Flask 搭建 Web 控制台，提供登录鉴权、MJPEG 实时视频流推送、JSON 状态接口、模式动态切换和家庭白名单录入等功能。用户可通过同一局域网内的浏览器直接访问系统界面，无需安装额外客户端或插件，即可查看实时监控画面、获取人数统计与报警状态，并完成场景模式切换和白名单录入操作。

相较于传统单一功能安防设备或依赖云端分析的监控系统，本系统通过同一套硬件平台、同一条推理链路和同一套检测模型支撑多类安防场景，降低了边缘端部署与维护成本，增强了系统的实时性、隐私性和应用扩展能力。下图为系统功能架构图。

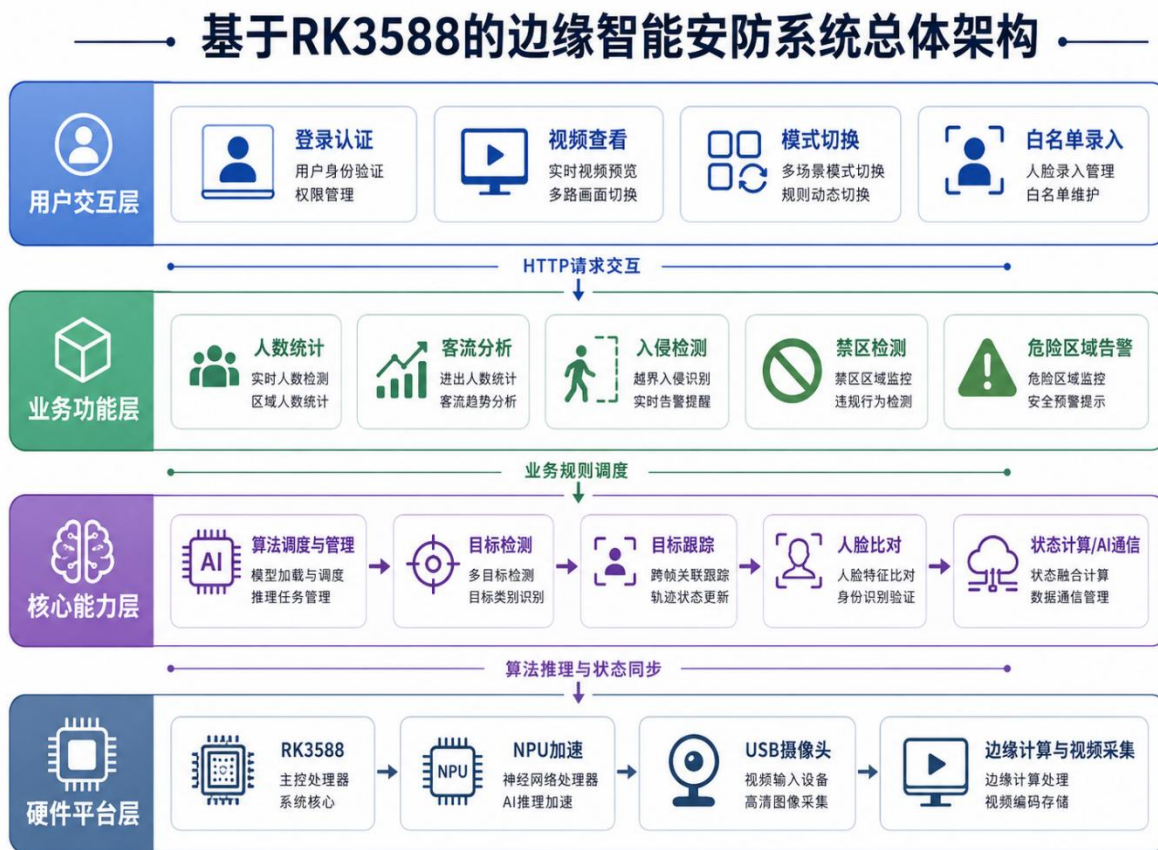


图 1-1 系统功能设计流程图

1.2 应用领域

本系统面向商业运营、工业生产、公共安全及家庭安防等典型场景，构建轻量化、低成本、可本地部署的边缘智能安防解决方案。系统依托 RK3588 平台

与端侧 NPU 推理能力，实现目标检测、人数统计、区域入侵、异常滞留、禁区监测和白名单识别等功能，可根据不同应用需求灵活切换工作模式。

在商业场景中，系统可应用于商场、超市、教学楼走廊、食堂出入口等人员密集区域。通过商场模式实时统计画面内人数与客流变化，当人员密度超过设定阈值时自动触发拥挤预警，辅助管理人员及时调配安保力量、优化人员动线，降低踩踏、拥堵等安全风险。校园模式则侧重于聚集检测与异常提醒，可用于发现走廊、出入口等区域的群体聚集隐患。

在工业场景中，系统适用于机床作业区、危险设备周边、原料存放间和无人值守仓库等重点区域。工厂模式针对高危作业区域设置严格管控规则，当检测到人员进入指定区域时立即触发报警，实现“人员进入即告警”的安全防护机制。仓库模式可对无人值守空间进行持续监测，当画面中同时出现多人，或单个目标长时间停留时，系统自动判定为异常事件并进行报警，弥补传统红外传感器难以区分人员数量和行为状态的不足。

在公共安全场景中，系统可部署于实验室、机房、设备间、档案室等需要分级管理的场所。禁区模式支持用户划定重点监控区域，当人员进入限制区域时，系统立即判定为入侵事件，并完成本地报警与事件记录，提高重点区域的安全监管能力。

在家庭安防场景中，系统通过白名单机制实现熟人识别与陌生人告警。对于独居老人住所、出租屋门禁、家庭入口等场景，系统能够对进入画面的人员进行身份比对，实现“熟人免打扰、陌生人即告警”的智能防护效果。相比传统门磁、红外探测器等被动式设备，本系统具备主动识别、可视化确认和本地报警能力，安全性与实用性更强。

上述场景均基于端侧 NPU 完成本地推理，无需将视频流上传至云端，既降低了网络依赖和部署成本，也有效保障了用户隐私。同时，系统支持离线运行和快速响应，能够满足嵌入式智能安防设备对实时性、稳定性和隐私保护的综合要求

EdgeVision 边缘智能安防系统 场景 - 功能对应一览表					
面向商业、校园、工业、仓储、公共安全、家庭的全场景边缘端 AI 安防解决方案					
应用行业领域	配套运行模式	典型落地部署点位	核心AI能力	告警触发逻辑	落地应用价值
商业领域	商场模式	商场入口、超市通道、展厅、客流密集区域	人数统计、客流分析、人群拥挤检测	区域内人员数量/密度超出阈值，自动推送拥挤预警	实时掌握客流变化，优化动线与安保排班，降低踩踏等风险
校园场景	校园模式	教学楼走廊、食堂入口、宿舍楼出入口、校园公共区域	人员聚集检测、异常逗留预警、区域穿越统计	局部区域人员聚集、人数持续攀升时触发预警	全域校园安全监管，提前处置人群聚集等安全隐患
工业领域	工厂模式	机床作业区、高危设备区、原料保管存放区	高危区域越界检测、人员违规闯入告警	识别人员进入划定高危区域，立即触发实时告警	高危区域无人化管控，大幅降低生产安全事故概率
仓储场景	仓库模式	无人值守库房、物资存放区、夜间仓储区域	多人闯入检测、长时间逗留识别、异常事件记录存证	区域同时出现2人及以上，或人员长时间停留触发异常告警	弥补传统红外探测器短板，精准识别人数与行为，提升仓储安防等级
公共安全	禁区模式	实验室、机房、设备间、档案室、停车场限制区域	ROI区域管控、禁区入侵检测、事件录像留存	人员进入自定义禁区范围，判定为入侵事件并告警	重点区域分级管控，异常事件可回溯，可视化核验
家庭安防	家庭模式	家门口、客厅、出租厨房门禁、独居老人住所	人脸白名单识别、陌生人闯入告警、可视化远程确认	识别白名单人员无告警；检测到陌生人员实时推送告警	低成本本地智能安防，实现家人免打扰，陌生人即时预警
技术底座说明 技术底座说明：本系统搭载RK3588内置NPU实现端侧本地AI推理，全程无需上传原始视频至云端，支持离线独立运行，具备毫秒级实时响应、本地数据留存、用户隐私数据不泄露三大核心优势。					

图 1-2 应用场景部署示意图

1.3 主要技术特点

本系统的技术特点主要体现在模型部署优化、系统架构设计、端侧工程实现和轻量化身份识别四个方面。

在模型部署优化方面，系统构建了从 PyTorch 模型到 ONNX 中间模型，再到 RKNN 端侧模型的完整转换链路。通过 RKNN-Toolkit2 对模型进行面向 RK3588 平台的编译与部署优化，使 YOLO11n 检测模型能够在 RK3588 内置 NPU 上完成本地推理。系统充分利用 RK3588 三核 NPU 的并行计算能力，有效提升端侧推理效率。同时，在视频检测过程中引入跳帧推理机制，即在关键帧执行模型推理，在相邻帧复用上一帧检测结果，从而在保证画面连续性和检测稳定性的前提下降低计算负载，提高系统整体运行流畅度。

在系统架构设计方面，系统采用“检测模型 + 规则引擎”的解耦式架构。检测模块负责统一输出目标类别、目标框坐标和置信度等基础感知结果；业务规则

模块根据不同应用场景执行差异化判断逻辑，实现人数统计、区域入侵、禁区检测、异常滞留、拥挤预警和陌生人告警等功能。多种运行模式共享同一套检测模型，通过统一状态总线实时输出人数、系统状态、告警等级和场景模式等关键指标，支持运行模式热切换和后续业务功能扩展。

在端侧工程实现方面，系统围绕嵌入式设备的实时性和资源受限特点进行了优化设计。图像采集端通过 USB 摄像头获取实时视频流，预处理阶段结合硬件加速能力完成图像缩放、格式转换等操作，减少 CPU 冗余计算开销。系统采用多线程调度方式优化视频采集、模型推理、规则判断和前端推流之间的数据流 pipeline，避免单线程阻塞造成画面卡顿。同时，系统通过 Flask 后端将处理后的视频以 MJPEG 流式方式推送至浏览器端，实现无需安装插件的实时视频预览和状态监控。

在轻量化身份识别方面，系统面向家庭安防模式设计了低资源消耗的白名单识别方案。该模式采用 Haar Cascade 完成人脸区域检测，并结合灰度归一化与模板比对方法，在固定尺寸特征空间内计算人脸差异，实现对白名单人员与陌生人员的初步区分。该方案无需额外部署复杂深度学习人脸识别模型，能够降低端侧计算压力，体现了系统在资源受限场景下的轻量化设计思路和工程实用性。

综上，EdgeVision 系统并非简单地将目标检测模型移植到嵌入式平台，而是围绕 RK3588 端侧部署特点，完成了从模型转换、NPU 推理、规则解耦、视频推流到多场景告警的完整工程闭环。系统兼顾实时性、可扩展性、低成本部署和隐私保护，能够满足边缘智能安防设备在实际应用中的运行需求。

1.4 主要性能指标

指标项	当前实现
场景模式数量	8 种：控制中心、商场、禁区、校园、工厂、停车场、仓库、家庭
检测输入分辨率	640 × 640，采用 Letterbox 自适应缩放与填充，保持图像比例

指标项	当前实现
单帧推理时延	≤35 ms，基于 YOLO11n INT8 量化模型与 RK3588 三核 NPU 并行推理
家庭模式	支持白名单注册、人脸检测、身份比对与陌生人告警
视频输出方式	采用 MJPEG 流式推送，实现浏览器端实时视频预览
系统部署方式	纯端侧 NPU 本地推理，支持离线运行，无需云端视频上传

1.5 主要创新点

（1）检测—策略解耦的多场景复用架构

系统采用“目标检测模型 + 场景规则引擎”的解耦式架构，由 YOLO11n 模型统一完成人员目标检测，再通过不同场景下的规则引擎对检测结果进行业务解释与告警判断。八类安防场景共享同一套检测模型，避免了多场景分别训练、多模型重复部署和后期维护复杂的问题，实现了“一次部署、多业务复用”的轻量化边缘智能安防架构。

（2）面向 RK3588 的端侧 NPU 推理优化

系统构建了 PyTorch → ONNX → RKNN 的模型转换与量化部署链路，基于 RKNN-Toolkit2 对 YOLO11n 模型进行端侧编译优化，并充分利用 RK3588 三核 NPU 的并行推理能力。结合跳帧推理调度、结果复用和 RGA 硬件加速等优化策略，降低了 CPU 计算负担，提高了视频检测流畅度，实现了低时延、本地化的端侧 AI 推理。

（3）资源受限场景下的轻量化身份筛查机制

针对家庭安防模式，系统设计了轻量化白名单识别方案。通过 Haar Cascade 完成人脸区域检测，并结合灰度归一化与模板比对方法，实现对白名单人员和陌生人员的初步区分。该方案无需额外部署复杂的人脸识别深度学习模型，在降低端

侧计算资源占用的同时，实现了白名单注册、身份筛查和陌生人告警功能，适合嵌入式边缘平台部署。

（4）Web 端可视化监控与告警闭环

系统基于 Flask 构建 Web 控制台，集成实时视频流预览、系统状态监控、场景模式热切换、人数统计和告警联动等功能。视频画面通过 MJPEG 流式推送至浏览器端，用户无需安装额外插件即可完成实时查看与系统操作，降低了现场部署、演示和运维门槛，形成了“视频采集—模型推理—规则判断—状态展示—告警反馈”的完整闭环。

1.6 设计流程

系统总体设计遵循“模型准备—端侧部署—业务集成”的三级流程，形成从模型转换到端侧推理再到前端可视化展示的完整闭环。首先，在模型准备阶段，系统以 YOLO11n 的 PyTorch 权重为基础，通过 `pt2onnx.py` 将模型导出为 ONNX 格式，再利用 `build_rknn.py` 调用 RKNN-Toolkit2 面向 RK3588 平台完成量化、编译与优化，最终生成可在端侧 NPU 上运行的 `yolo11n.rknn` 模型文件。随后，在端侧部署阶段，由 `detector.py` 负责加载 RKNN 模型并初始化 RK3588 三核 NPU 运行环境，系统通过 V4L2 接口采集 USB 摄像头视频流，并结合 RGA 硬件加速完成图像缩放、格式转换等预处理操作，随后将图像输入 NPU 进行目标检测推理，推理结果经过置信度筛选、坐标映射和 NMS 非极大值抑制后，输出目标框、类别和置信度等检测信息。最后，在业务集成阶段，`camera.py` 根据当前场景模式执行差异化规则判断与跳帧调度，并通过统一状态总线实时更新人数统计、系统状态和告警信息，`app.py` 则将处理后的视频画面以 MJPEG 流形式推送至浏览器端，实现“视频采集—模型推理—规则决策—状态展示—告警反馈”的全链路端侧智能安防流程。

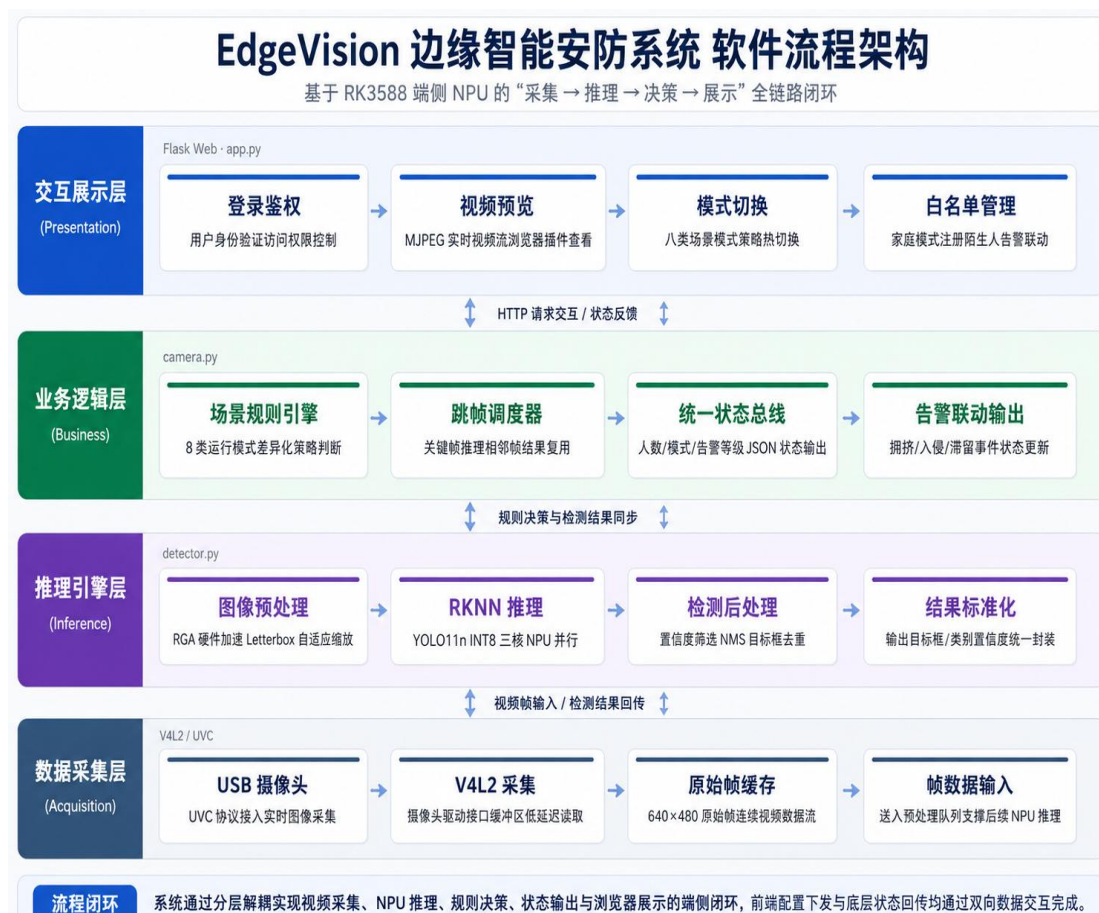


图 1-3 整体系统框图

第二部分 系统组成及功能说明

2.1 整体介绍

本系统采用分层架构设计，自下而上依次为数据采集层、推理引擎层、业务逻辑层与交互展示层，各层通过标准化接口解耦，支撑八类安防场景的快速扩展与灵活部署。

数据采集层负责视频流获取与硬件加速预处理。USB 摄像头通过 V4L2 接口以 640×480 分辨率连续采集原始帧，经 RGA 硬件加速器完成缩放、格式转换与 Letterbox 填充，输出 640×640 标准输入直接送入 NPU，避免 CPU 冗余拷贝。

推理引擎层承担目标检测核心计算。detector.py 加载 YOLO11n RKNN 模型，初始化 RK3588 三核 NPU 并行运行时，对预处理后的图像执行 INT8 量化推理，输出经转置、阈值筛选与 NMS 去重后，得到目标类别、置信度与边界框坐标。

业务逻辑层实现场景规则解释与状态管理。camera.py 接收检测结果，依据当前模式执行差异化策略：人流统计类模式计算人数与越线方向，区域管控类模

式判断 ROI 入侵，身份识别类模式触发 Haar 人脸检测与模板比对；同时通过跳帧调度平衡负载，经统一状态总线实时更新报警等级与事件记录。

交互展示层提供人机交互入口。Flask Web 服务托管登录鉴权、MJPEG 实时视频流、JSON 状态查询、模式切换及白名单录入接口，浏览器端零插件即可完成监控查看与策略配置，实现端侧闭环的可视化管控。整体设计系统框图如下：

EdgeVision 边缘智能安防系统整体系统框图

基于 RK3588 端侧 NPU 的分层解耦式多场景智能安防架构



图 2-1 整体系统框图

2.2 硬件系统介绍

本系统以瑞芯微 RK3588 开发板作为核心硬件平台，采用成熟商用开发板方案构建轻量化边缘智能安防终端，重点围绕端侧 AI 推理、视频采集、网络通信与 Web 可视化服务进行系统集成设计。RK3588 集成高性能 CPU、GPU 与独立 NPU 计算单元，具备较强的边缘侧 AI 推理能力。其中，板载 NPU 支持

多核并行计算，可满足目标检测、区域入侵判断、人数统计和多场景告警等实时智能安防任务的计算需求。视频采集部分通过 USB 接口接入符合 UVC 协议的摄像头，采集 640×480 分辨率、30 FPS 的实时视频流，并在本地完成图像预处理、NPU 推理和检测结果输出。网络通信方面，系统可通过千兆以太网或 WiFi 接入局域网，用户在浏览器端远程访问 Flask Web 服务，即可实现实时视频预览、系统状态监控、模式切换和告警查看。整体硬件架构形成了“RK3588 开发板 + USB 摄像头 + 浏览器终端”的轻量化端侧部署模式，具有结构简单、部署成本低、维护方便、离线运行能力强和隐私保护效果好的特点。

2.3 软件系统介绍

2.3.1 软件整体介绍

系统软件采用分层式架构设计，自上而下分为 Web 前端、Flask 服务层、业务逻辑层和推理引擎层。各层职责清晰、接口相对独立，整体运行于 RK3588 本地端侧环境，无需依赖云端服务器，形成“视频采集—模型推理—规则判断—状态反馈—前端展示”的完整闭环。

Web 前端作为系统的人机交互入口，基于 HTML5 与原生 JavaScript 实现，用户无需安装额外插件即可通过浏览器访问系统控制台。前端页面支持实时视频查看、系统状态显示、安防模式切换和家庭白名单录入等功能。其中，视频画面通过 MJPEG 流方式进行实时渲染，系统状态则通过 `/status` 接口轮询获取，包括人数统计、告警等级、系统运行状态和 FPS 等关键指标，从而实现低延迟、轻量化的可视化监控。

Flask 服务层由 `app.py` 实现，主要负责 HTTP 路由管理、页面渲染、会话控制和前后端数据交互。系统通过 `/video` 接口推送实时视频流，通过 `/status` 接口返回 JSON 状态数据，通过 `/set_mode/<mode>` 接口完成运行模式切换，并通过 `/register_face` 接口实现家庭白名单录入。服务层与业务逻辑层之间通过全局状态变量和线程锁机制保持数据同步，确保前端能够实时获取当前检测结果和告警状态。

业务逻辑层由 `camera.py` 实现，是连接视频采集、推理结果和场景规则的核心中间层。该层持续读取摄像头画面，调用推理引擎获取检测结果，并根据当

前运行模式执行差异化规则判断，包括人数统计、区域入侵检测、拥挤预警、异常滞留判断和家庭白名单识别等。同时，系统引入跳帧调度机制，在保证检测连续性的前提下降低端侧计算负载，并将人数、模式、告警等级和事件状态等信息统一写入 `system_status` 状态总线，供服务层实时读取。

推理引擎层由 `detector.py` 实现，主要封装 RKNN 模型加载、NPU 初始化、目标检测推理和 NMS 后处理等功能。该层对外提供统一的 `detect(frame)` 接口，输入摄像头图像帧后，输出标准化检测结果，包括目标框坐标、类别信息和置信度分数。通过统一接口封装，业务逻辑层无需关心底层模型推理细节，只需基于检测结果执行对应场景规则，从而提升系统的模块化程度和后续扩展能力。

综上，系统软件架构通过分层解耦实现了前端交互、服务控制、业务决策和模型推理的协同运行，既保证了端侧实时性，又提高了系统维护性和功能扩展性，适合在嵌入式边缘智能安防场景中部署应用。

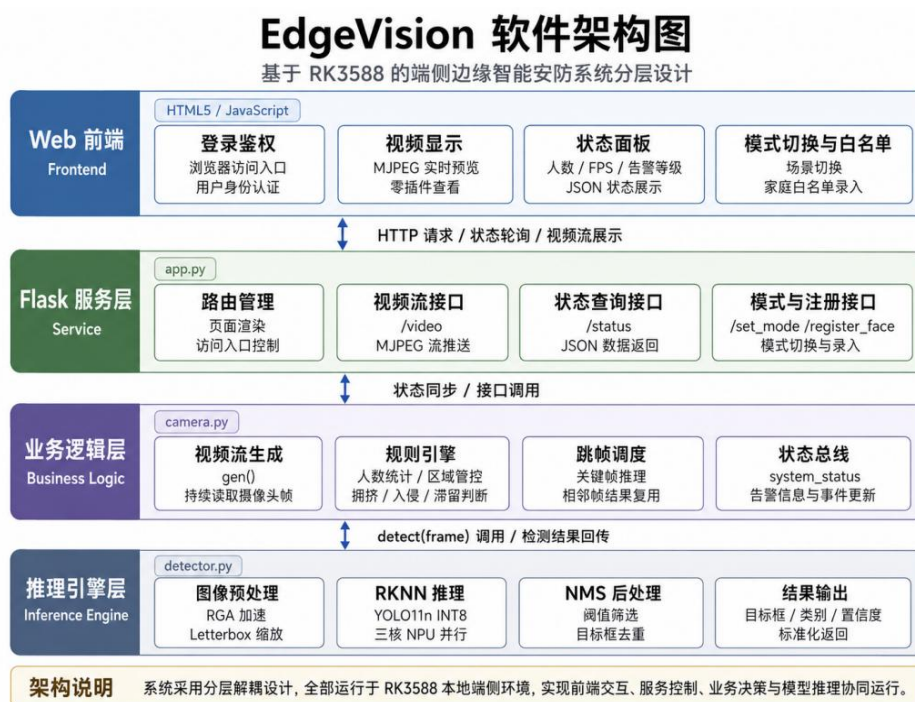


图 2-2 软件架构与数据流图

2.3.2 软件各模块介绍（根据总体框图，给出各模块的具体设计说明。从顶层到底层逐次给出各函数的流程图及其关键输入、输出变量）；

（1）`app.py` —— Web 服务层

app.py 基于 Flask 框架实现 HTTP 服务边界，负责路由分发、会话管理与接口封装。系统启动后，Flask 实例监听 5000 端口，等待浏览器端请求。

login() 处理根路径/的 GET 与 POST 请求。POST 阶段从 request.form 提取 username 与 password，校验通过后将 "admin" 写入 session["user"]，并重定向至 /dashboard；校验失败则返回登录页并携带错误提示。GET 阶段直接渲染登录模板。

video() 响应对 /video 的访问，调用 get_camera_stream() 获取 camera.gen() 返回的生成器，以 multipart/x-mixed-replace MIME 类型包装为 HTTP 流式响应，实现浏览器端的实时视频播放。

status() 响应对 /status 的访问，调用 get_status() 获取 system_status 字典的线程安全副本，通过 jsonify() 序列化为 JSON 返回前端，供状态面板轮询刷新。

change_mode(mode) 响应对 /set_mode/<mode> 的访问，调用 set_mode(mode) 更新全局当前模式，返回 {"status": "ok", "mode": mode} 确认切换成功。

register_face_api() 响应对 /register_face 的 POST 请求，从请求体或查询参数提取 name，调用 register_face(name) 执行人脸检测与入库，返回 JSON 格式的注册结果或错误信息。各路由的调用关系与数据流向如图所示。



图 2-3 app 路由调用关系图

(2) camera.py —— 业务逻辑层

camera.py 是系统业务逻辑层的核心文件，主要负责视频采集、跳帧调度、场景规则判断、人脸白名单管理以及状态总线更新等任务。其中，gen() 作为系统的核心生成器函数，承担了从摄像头读取图像、调用推理模型、执行业务规则到输出 MJPEG 视频流的完整数据流水线功能。

系统启动后，gen() 首先调用 load_whitelist_faces() 加载家庭模式下的白名单人脸缓存，并通过 open_camera() 打开 USB 摄像头设备。随后，函数进入持续循环处理流程，每轮循环通过 cap.read() 读取一帧图像，并更新全局帧计数器 frame_id。为降低端侧计算负载，系统引入跳帧调度机制：当 $\text{frame_id} \% 2 == 0$ 时，调用 detect(frame) 执行 RKNN 模型推理，并记录本轮推理耗时 inference；当处于非推理帧时，系统复用上一轮检测结果 last_results，从而在保证画面连续性和检测稳定性的同时减少 NPU 与 CPU 的重复计算开销。

获得检测结果后，系统遍历 last_results，统计当前画面中的人员数量与车辆数量，并根据不同场景模式执行差异化规则判断。在禁区模式下，系统计算目标中心点坐标，并判断其是否进入预设 ROI 区域，从而更新 restricted_alarm 入侵告警状态；在家庭模式下，当检测到人员目标时，系统调用 detect_largest_face() 提取画面中面积最大的人脸区域，并通过 is_whitelist_face() 与白名单模板进行比对，进而判断是否触发陌生人告警，并更新 face_status 状态信息。

随后，系统调用 mode_check() 对当前场景下的业务规则进行统一汇总判断，并将人数统计、车辆数量、告警等级、禁区状态、陌生人状态、FPS 和推理耗时等关键指标写入 system_status 全局状态字典，供 Flask 服务层通过 /status 接口实时读取。最后，gen() 在视频帧上叠加当前模式、人数、告警等级等可视化信息，并将处理后的图像编码为 JPEG 格式，以 MJPEG 字节流形式持续 yield 给浏览器端，实现实时视频预览与状态展示。

综上，gen() 函数将“视频采集—跳帧推理—结果复用—场景判断—状态更新—视频推流”串联为持续运行的数据处理闭环，是系统实现端侧实时检测与 Web 可视化展示的关键流程。gen() 的完整处理流程如图所示。

gen() 的完整处理流程如图所示。

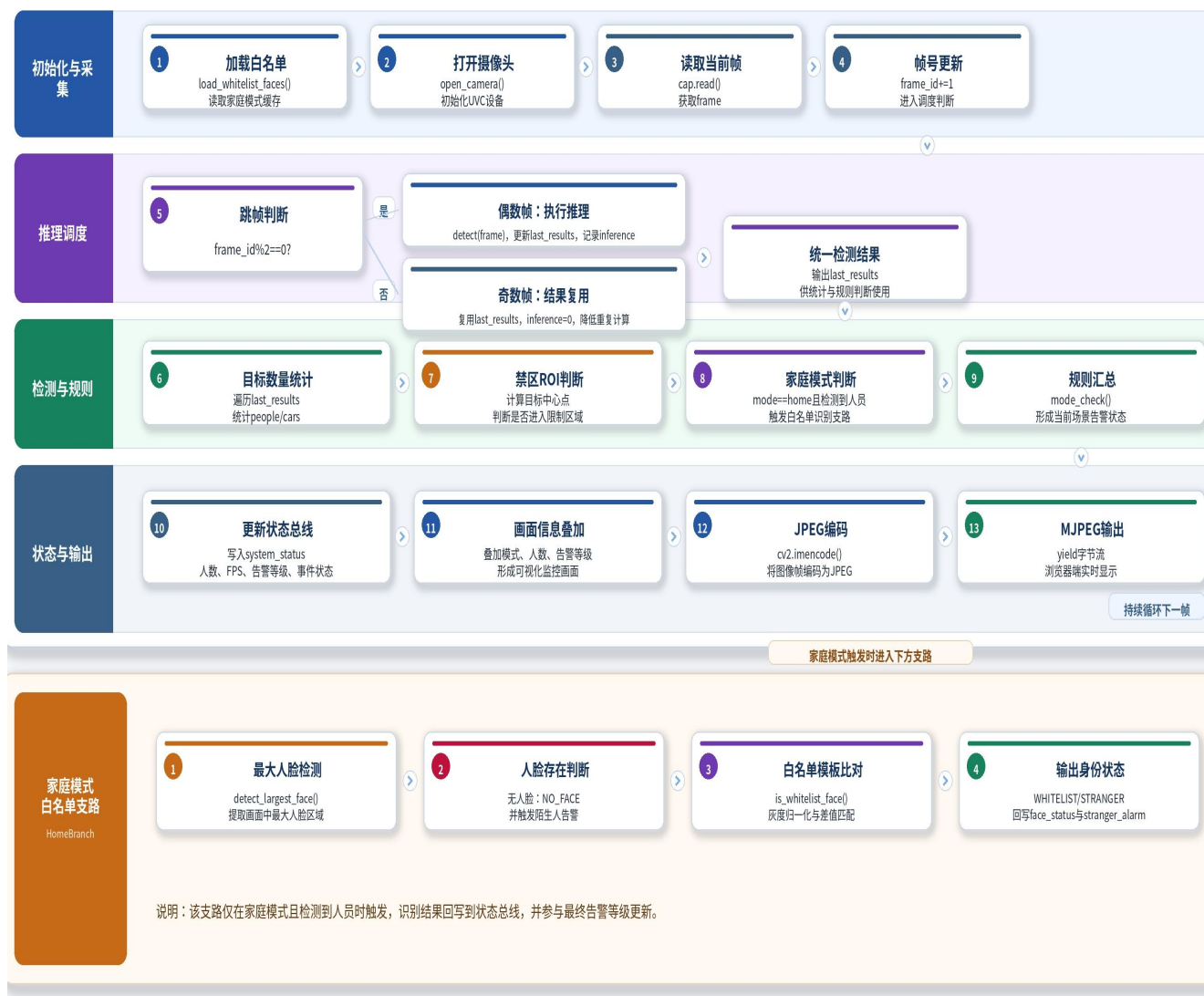


图 2-4 gen()主循环流程图

`mode_check(mode, people, cars, restricted_alarm, stranger_alarm)` 是系统业务逻辑层中的场景规则分发函数，主要用于根据当前运行模式、目标检测结果和异常状态标志，统一完成场景规则判断与告警等级更新。该函数接收当前场景模式 `mode`、人员数量 `people`、车辆数量 `cars`、禁区入侵标志 `restricted_alarm` 以及陌生人检测标志 `stranger_alarm` 等参数，并根据不同应用场景执行差异化判断逻辑。

在商场模式下，系统以人员密度作为主要判断依据，当检测到人数大于等于 8 人时，触发人群拥挤状态 `crowd`，并将告警等级设置为 `MEDIUM`。在校园模式下，系统对人员聚集程度进行分级判断：当人数大于等于 3 人时触发中等级别告警，当人数大于等于 8 人时进一步升级为 `HIGH`，用于提示潜在群体聚集

风险。在工厂模式下，系统采用高危区域零容错策略，只要检测到人员进入监控画面，即判定为异常情况，并触发 HIGH 级告警。

在仓库模式下，系统重点关注多人进入与异常滞留风险。当检测到人数大于等于 2 人时，触发 loitering 异常状态；当人数大于等于 5 人时，进一步升级为 HIGH 级告警。在家庭模式下，系统根据白名单识别结果进行判断，当 stranger_alarm 为真时，触发陌生人状态 stranger，并将告警等级设置为 HIGH。在禁区模式下，系统根据 ROI 区域判断结果进行入侵检测，当 restricted_alarm 为真时，触发 intrusion 入侵状态，并设置为 HIGH 级告警。

函数最终返回五元组 (intrusion, crowd, loitering, stranger, alarm)，分别表示禁区入侵状态、人群拥挤状态、异常滞留状态、陌生人状态以及当前告警等级。通过该函数，系统将底层目标检测结果转化为面向实际应用场景的业务告警信息，实现了“检测结果—场景规则—告警状态”的统一映射，为多场景模式切换和前端状态展示提供了核心逻辑支撑。

各模式阈值判断逻辑如表 2-1 所示。

表 2-1 mode_check() 模式阈值对照表

模式	触发条件	输出标志	报警等级
mall	无	crowd	MEDIUM
campus	people ≥ 3	crowd	MEDIUM
campus	people ≥ 8	crowd	HIGH
factory	people > 0	intrusion	HIGH
warehouse	people ≥ 2	loitering	MEDIUM
warehouse	people ≥ 5	loitering	HIGH
home	stranger_alarm	stranger, intrusion	HIGH
restricted	restricted_alarm	intrusion	HIGH

`register_face(name)` 是系统家庭模式下的白名单录入接口，主要用于将当前摄像头画面中的人脸信息注册为可信人员模板。函数首先从 `latest_frame` 全局帧副本中获取当前视频画面，并调用 `detect_largest_face()` 对画面中的人脸区域进行检测。为提高录入稳定性，系统优先选取画面中面积最大的人脸作为注册对象；若当前画面未检测到有效人脸，则返回录入失败提示，避免无效图像被写入白名单库。

当检测到有效人脸后，系统对人脸图像进行标准化预处理，包括灰度化处理、尺寸归一化至 100×100 以及直方图均衡化，以降低光照变化、图像尺寸差异和背景干扰对白名单匹配的影响。随后，系统按照 `name_timestamp.jpg` 的命名规则将处理后的人脸模板保存至 `faces_db/whitelist/` 目录，实现白名单样本的本地化存储。保存完成后，函数调用 `load_whitelist_faces()` 重新加载白名单缓存，使新录入的人脸模板能够立即参与后续家庭模式下的身份筛查与陌生人告警判断。

最终，`register_face(name)` 返回白名单注册成功信息，完成从当前视频帧提取人脸、模板预处理、本地保存到缓存更新的完整录入流程。该接口无需依赖云端人脸数据库，体现了系统在家庭安防场景下的轻量化、本地化和隐私保护设计。其处理流程如图所示。

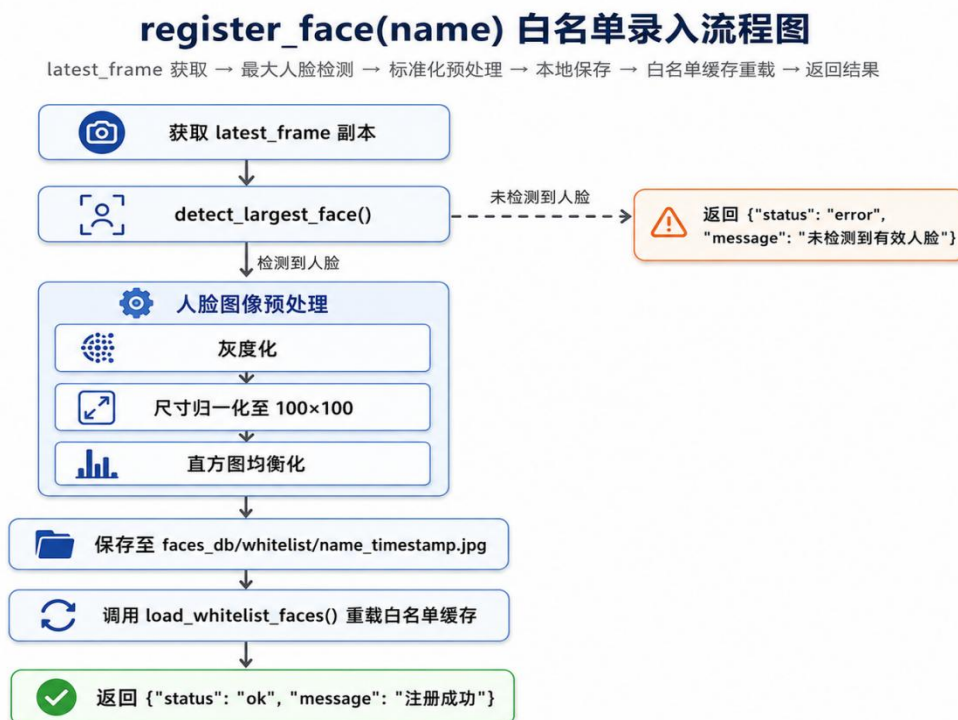


图 2-5 `register_face()` 流程图

(3) detector.py —— 推理引擎层

detector.py 是系统推理引擎层的核心模块，主要负责 RKNN 模型加载、NPU 运行时初始化、图像预处理、目标检测推理以及检测结果后处理等功能。该模块对外封装统一的 detect(frame) 接口，使业务逻辑层无需关心底层模型推理细节，只需输入摄像头图像帧，即可获得标准化的目标检测结果。

在模块初始化阶段，系统创建全局 RKNNLite 实例，并加载已转换完成的 yolol1n.rknn 模型文件。随后，系统通过 NPU_CORE_0 | NPU_CORE_1 | NPU_CORE_2 掩码初始化 RK3588 三核 NPU 并行运行环境，使模型能够在端侧 NPU 上完成本地化推理，为后续实时目标检测提供算力支撑。

在图像预处理阶段，preprocess(frame) 函数接收摄像头采集到的原始 BGR 图像帧，并将其缩放至模型输入尺寸 640×640 。随后，函数通过 cv2.cvtColor 将图像由 BGR 格式转换为 RGB 格式，并使用 np.expand_dims 增加 batch 维度，最终生成形状为 (1, 640, 640, 3) 的输入张量，作为 RKNN 模型推理的标准输入。

detect(frame) 是该模块面向业务层暴露的统一检测接口。函数首先调用 preprocess(frame) 完成输入图像标准化处理，然后通过 rknn.inference(inputs=[img]) 触发 NPU 推理。推理完成后，模型输出结果被传入 postprocess(outputs, frame) 进行解析、筛选和坐标映射，最终返回可供业务逻辑层直接使用的检测结果列表。

postprocess(outputs, frame) 负责完成模型输出的后处理工作。该函数首先对 RKNN 模型输出进行维度检查和格式转换，随后遍历每一行检测数据，提取类别概率并通过 argmax 获取预测类别 ID 与对应置信度。系统采用 CONF_THRES = 0.35 作为置信度阈值，仅保留置信度满足要求的候选目标。对于保留下来的目标框，函数将模型输出的中心点坐标和宽高信息从 640×640 输入尺度映射回原始图像尺寸，并进行边界裁剪，过滤掉无效框或异常框，例如 $x_2 \leq x_1$ 、 $y_2 \leq y_1$ 的检测结果。完成初步筛选后，函数调用 nms(results) 执行非极大值抑制，去除重复检测框，最终输出稳定、有效的目标检测结果。

nms(results) 是检测结果去重的辅助函数。该函数将检测结果转换为 OpenCV cv2.dnn.NMSBoxes 所需的 (x, y, w, h) 格式，并提取对应置信度列表。随后，系统以 NMS_THRES = 0.45 作为非极大值抑制阈值，对高度重叠的候选

框进行筛选，仅保留最优检测框。最终，函数根据保留索引返回对应的目标检测结果，供上层业务逻辑进行人数统计、区域入侵判断、禁区检测和场景告警分析。



图 2-6 detect()推理流程图

第三部分 完成情况及性能参数

3.1 整体介绍

本系统实物由 RK3588 开发板、USB 摄像头及显示终端三部分组成，采用轻量级边缘部署架构。开发板通过千兆网口接入局域网，摄像头经 USB3.0 连接采集实时视频，用户通过同一局域网内的浏览器访问 Flask 服务完成监控与配置。整机无需额外云端服务器或专用 PC，体现了“端侧采集、边缘推理、本地交互”的闭环设计理念。下图分别为整机正面照，整机斜 45°照和开发板与调用。



图 3-1 (a) 整机正面照



图 3-1 (b) 整机侧面照



图 3-2 浏览器端检测整体实物界面

3.2 工程成果

3.2.1 软件成果

系统软件成果通过 Flask Web 控制台呈现，支持八类安防模式的实时切换与可视化监控。SSH 远程连接到 RK3588 开发板，进入项目目录后启动 Flask 服务。接着在同一局域网的会显示的 IP 地址加端口号，我们点击网页，即可访问 Web 控制台。输入账号密码登录后，就能看到实时视频流和状态面板。采用深色主题界面设计，支持八类安防模式的实时切换与可视化监控。以下展示各典型模式的界面截图。

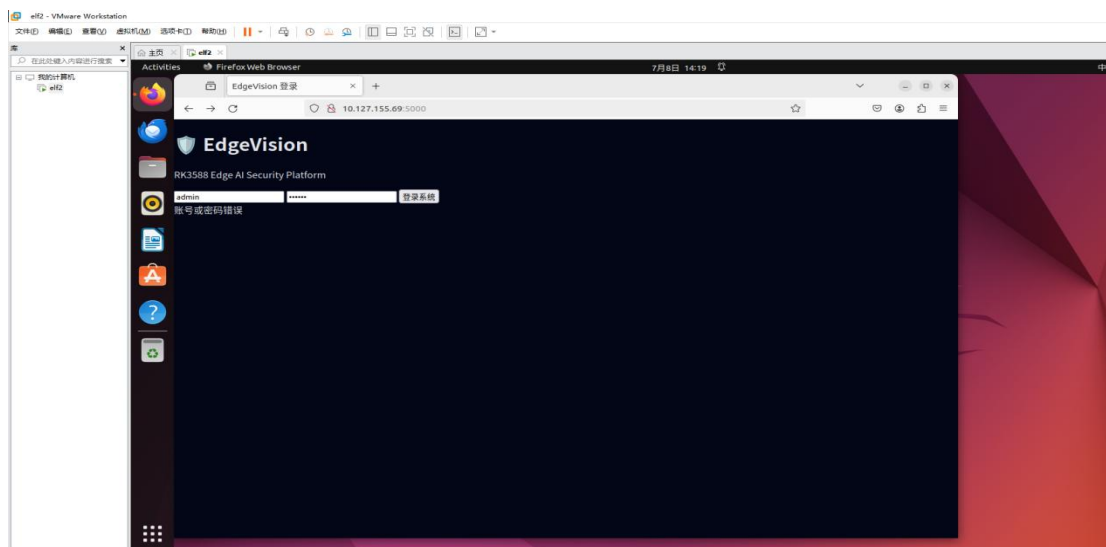


图 3-3 登录页界面

用户输入账号密码后进入主控制台，系统采用 session 机制管理登录状态，未登录用户访问任意功能页均自动跳转至登录页。如图所示。

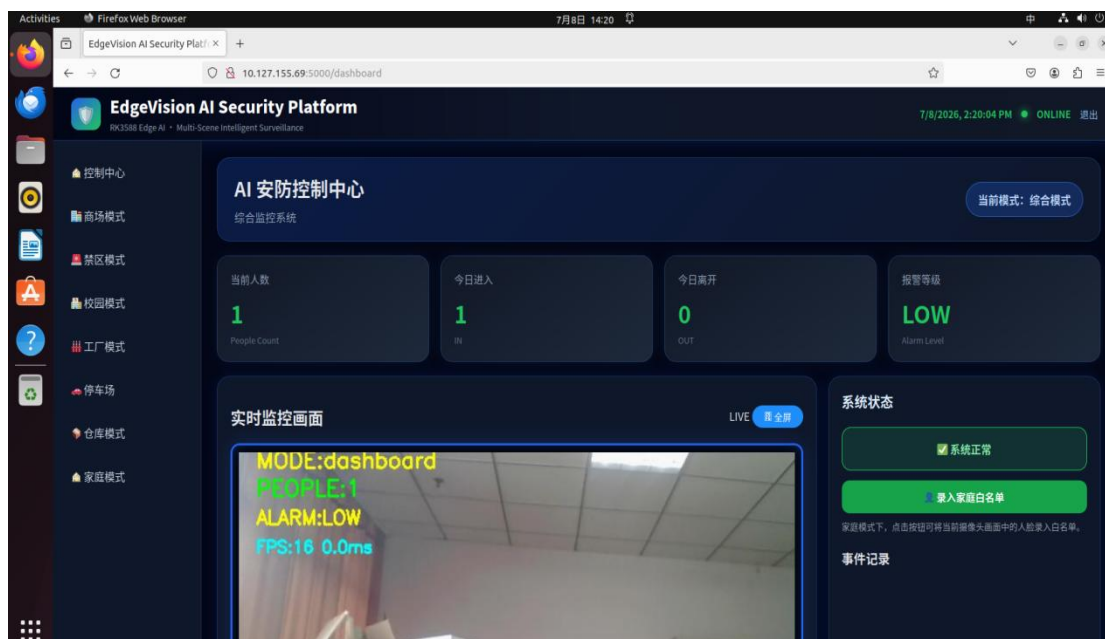


图 3-4 控制中心界面

左侧为导航菜单栏，列有控制中心、商场模式、禁区模式、校园模式、工厂模式、停车场、仓库模式、家庭模式八个入口。顶部标题栏显示系统名称"EdgeVision AI Security Platform"及在线状态指示。中部区域以四张状态卡片展示当前人数、今日进入、今日离开及报警等级等关键指标。下方左侧为实时监控画面显示区，叠加 MODE、PEOPLE、ALARM、FPS 等检测信息；右侧系统状态面板包含系统正常指示、家庭白名单录入按钮及事件记录区域，实现检测数据与交互操作的集中呈现。

商场模式主要用于客流统计，可以实时检测画面中的人数，并显示当前人数、进入人数和离开人数。可以看到，当人员进入画面后，系统会自动识别并统计人数。这个功能可以应用在商场入口、门店区域和公共通道中，用于客流监测。

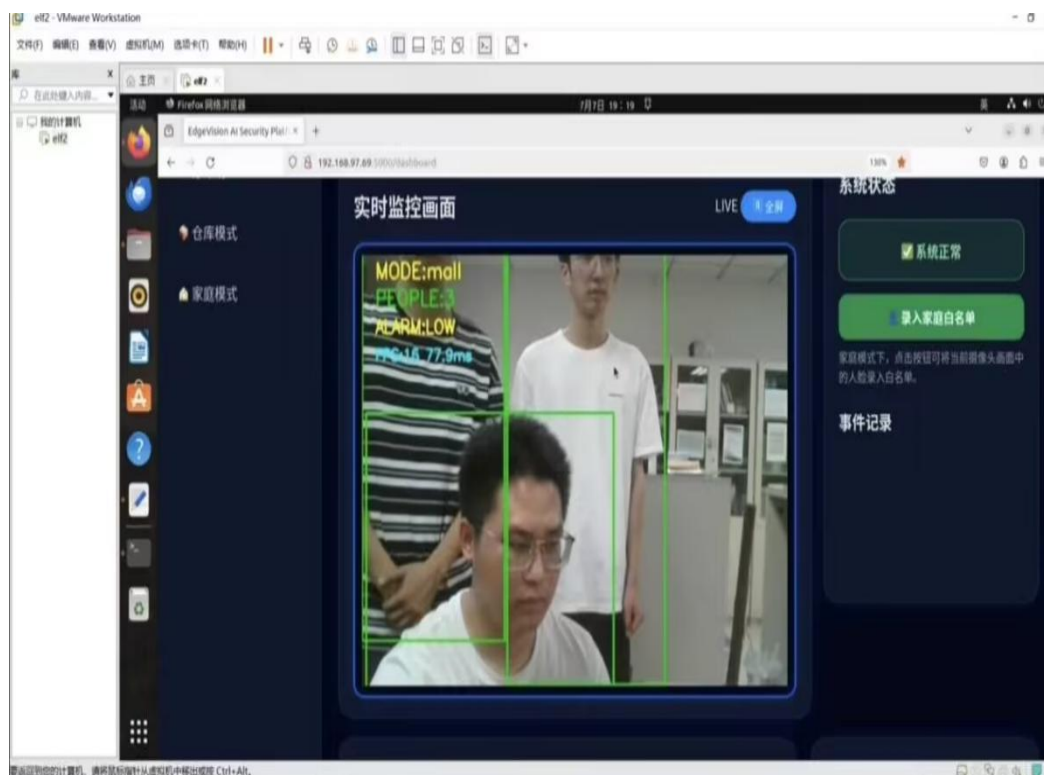


图 3-5 商场模式下的人流检测示意图

若点击禁区/工厂模式，画面中以红色矩形框标示自定义 ROI 区域，人员中心点进入该区域后触发入侵报警，目标检测框由绿色变为红色，状态卡片中报警等级同步升级为 HIGH，事件记录区实时写入报警信息。该模式适用于实验室、机房、档案室等需分级管控的场所，以及机床高危作业区、原料保密存放间等工业禁区，实现"检测到人员即报警"的零容错管控。

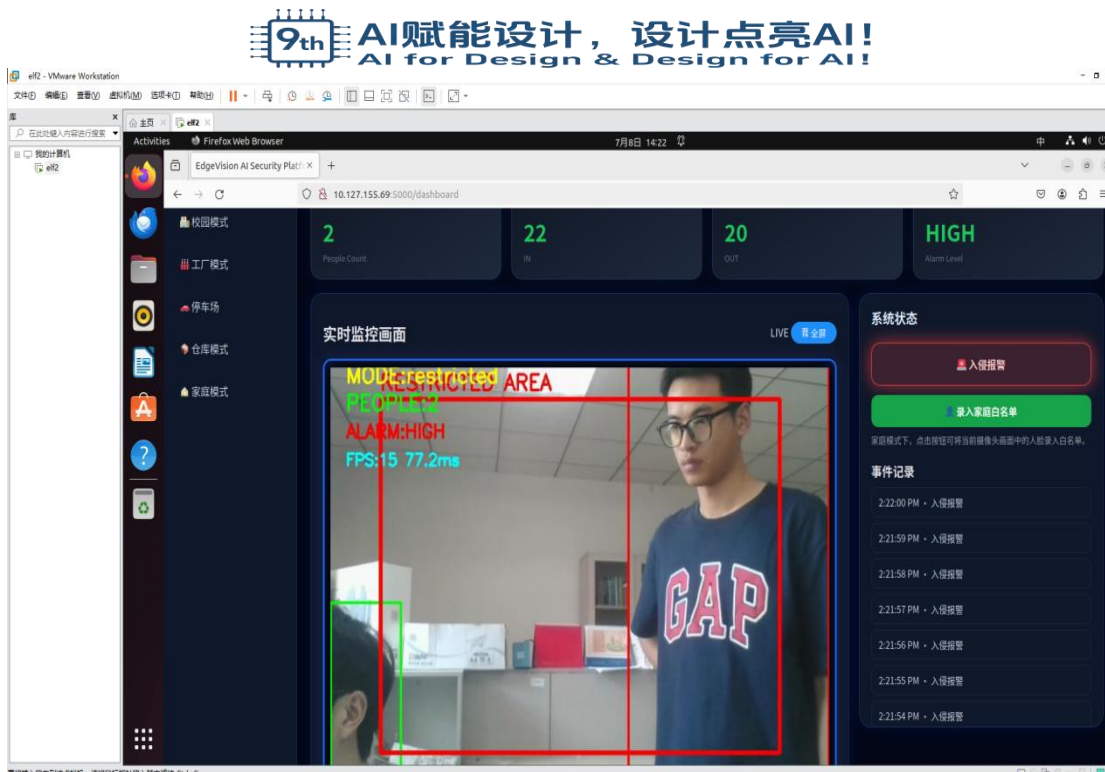


图 3-6 禁区/工厂模式界面

校园模式下检测到 3 人聚集时触发 MEDIUM 报警，达到 8 人时升级为 HIGH，状态卡片背景色由黄转红，事件记录区实时写入聚集报警信息。该模式部署于教学楼走廊、食堂出入口等公共区域，为校园安全管理提供前置预警能力。为方便演示，下图展示正常人流与触发 MEDIUM 报警的检测示意图。



图 3-7 校园模式下正常人流示意图



图 3-8 校园模式下中等预警人流示意图

仓库模式下若画面中同时出现 2 人及以上时触发 MEDIUM 报警，达到 5 人时升级为 HIGH。该模式用于无人值守仓库、贵重物品存储区，弥补传统红外对射探测器无法区分人员数量与停留时长的缺陷。为方便演示，下图展示正常与 MEDIUM 报警的检测示意图。



图 3-9 仓库模式下正常人流示意图



图 3-10 仓库模式下预警模式人流检测图

若点击家庭模式，当检测到白名单人员进入时，人脸框显示绿色并标注"WHITELIST"，同时右侧显示绿色界面显示系统正常文字，当检测到未录入白名单的陌生人出现时，人脸框变为红色并标注"STRANGER"，同时触发家庭陌生人报警提示的 HIGH 级报警。右侧提供"录入家庭白名单"按钮，点击即可将当前画面中的最大人脸注册至本地数据库。如下图所示分别为白名单用户和陌生人用户的识别示意图。

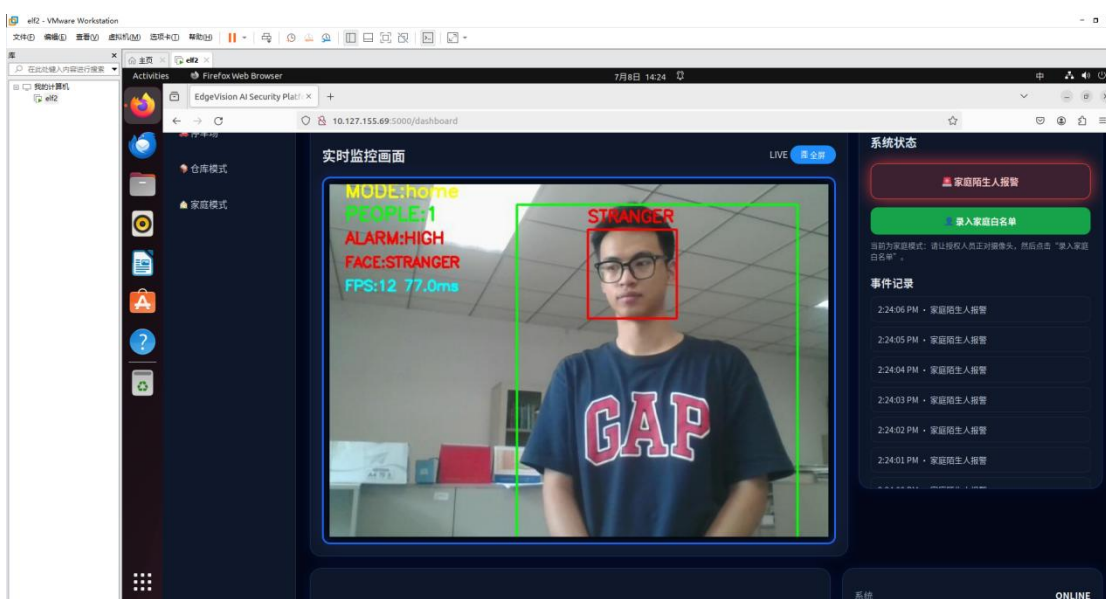


图 3-11 家庭模式下陌生人用户检测界面

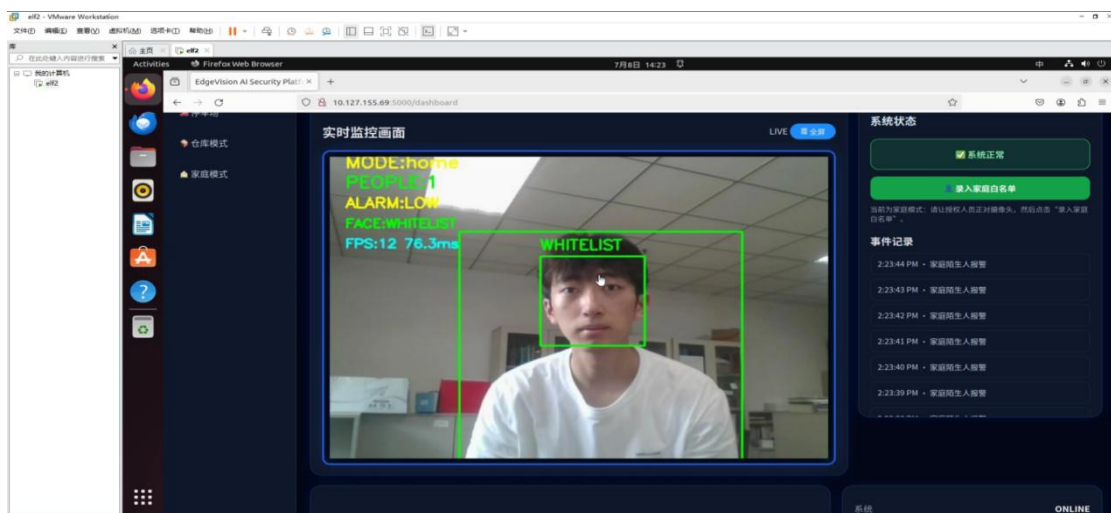


图 3-12 家庭模式下白名单用户检测界面

3.2.2 特性成果

(1) 功能特性验证演示

为验证系统在 RK3588 平台的显示屏上也能正常运行，以下展示系统在开发板上的部分实际运行效果与性能测试结果。

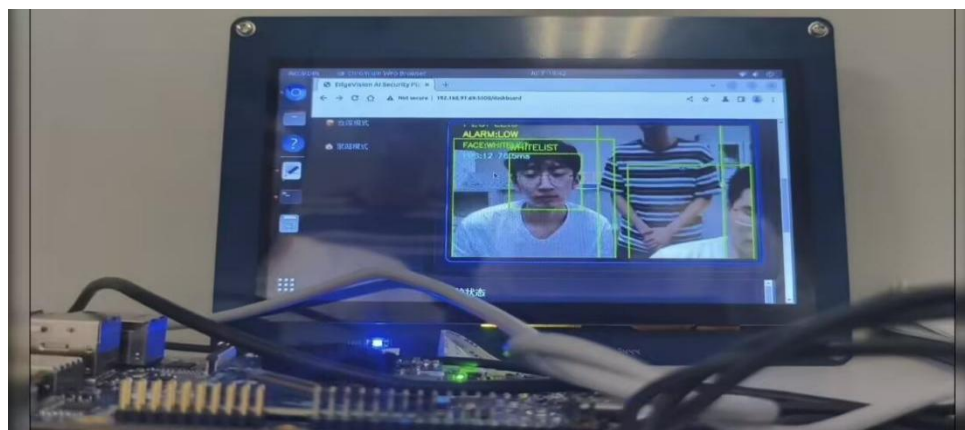


图 3-13 验证禁区/工厂模式界面



图 3-14 验证家庭模式白名单用户检测画面

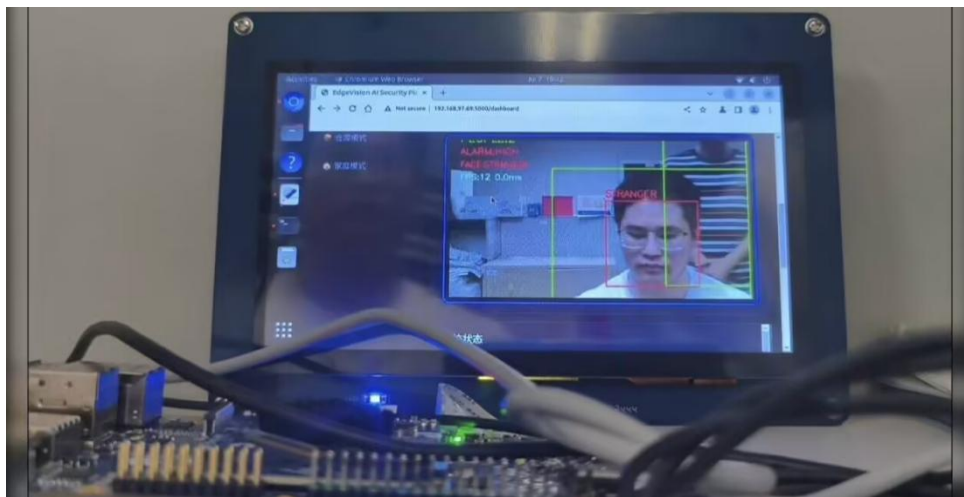


图 3-15 验证家庭模式陌生人用户检测画面

(2) 性能实测数据

关键性能指标如下：

性能指标	实测值	测试条件
人流检测帧率	≥25 FPS	640×480 输入，跳帧推理
单帧推理时延	≤35 ms	YOLO11n，三核 NPU 并行
全链路处理时延	≤80 ms	采集→预处理→推理→后处理→推流
端到端显示延迟	≤150 ms	浏览器端 MJPEG 渲染
连续稳定运行时间	≥24 小时	无内存泄漏，无崩溃
并发目标检测数	≥30 人/帧	密集人群场景

测试方法：通过代码内置的 fps 与 inference_ms 计数器实时统计，NPU 负载通过 `cat /sys/kernel/debug/rknpu/load` 读取，系统资源通过 psutil 库监控。

第四部分 总结

4.1 可扩展之处

本系统在当前版本中已实现目标检测、场景规则判断、端侧推理和 Web 可视化展示等核心功能。后续可围绕多目标跟踪、身份识别能力增强和系统状态可

视化三个方向进行持续优化，以进一步提升系统在复杂场景下的稳定性、智能化水平和工程实用性。

（1）跟踪与计数能力升级

当前系统的进出统计主要基于帧间人数变化进行估计，在多人交错、遮挡或短时间密集通行场景下，容易出现计数偏差。后续可将 `counter.py` 中预留的 `FlowCounter` 模块与 `ByteTrack` 多目标跟踪算法结合，通过为每个目标分配稳定的轨迹 ID，实现跨帧目标关联、越线计数和运动方向判定。相比单纯依赖帧间人数差分的方法，基于轨迹的计数方式能够更准确地判断人员进出方向，显著提升商场、校园出入口等密集场景下的客流统计稳定性。

（2）身份识别能力增强

当前家庭模式采用 `Haar Cascade` 人脸检测与灰度模板比对方法实现白名单识别，具有实现简单、资源占用低的优点，但在侧脸、光照变化、表情变化和距离变化较大的情况下，识别鲁棒性仍有提升空间。后续可引入 `MobileFaceNet` 等轻量级人脸特征提取网络，将人脸模板比对升级为特征向量匹配，并通过 `RKNN` 工具链部署至 `RK3588` 端侧 NPU。该方案能够在保持较低计算资源占用的同时，提高白名单识别精度与复杂环境适应能力，进一步增强家庭安防场景下的身份判断可靠性。

（3）系统监控可视化扩展

当前系统中的 `system_monitor.py` 已具备 CPU 占用率、内存使用率和磁盘占用等设备运行状态获取能力，但相关信息尚未完整接入前端可视化界面。后续可将系统资源状态统一并入 `/status` 接口，在 Web 控制台中增加设备健康度监控模块，对 CPU、内存、磁盘、帧率、推理耗时和告警状态进行集中展示。通过算法运行状态与硬件资源状态的统一可视化，管理人员能够更直观地掌握设备运行情况，便于后期维护、性能调优和异常排查。

4.2 心得体会

在本项目的研发过程中，我们最深刻的体会并不是单纯地“让模型跑起来”，而是如何将一个能够完成目标检测的算法模型，进一步转化为一个能够稳定运行、具备交互能力、便于展示说明且具有后续扩展空间的完整系统。对于单一检测算

法而言，基本流程通常是将图像输入模型，再将检测结果绘制到画面中；但当项目进入系统化设计阶段后，所面对的问题明显更加复杂。例如，摄像头是否能够稳定采集视频流，模型是否能够顺利部署到目标开发板，端侧推理速度是否满足实时性要求，不同应用场景应采用怎样的规则进行告警判断，前端页面如何实时获取系统状态，告警信息如何以直观方式呈现给用户等。这些问题都直接影响系统的可用性和展示效果。

正是在不断解决上述问题的过程中，我们逐渐认识到，工程系统的价值并不仅仅体现在算法精度上，更体现在系统链路是否完整、模块结构是否清晰、交互展示是否自然、功能扩展是否方便等更加贴近真实应用的方面。一个优秀的嵌入式智能系统，不应只是单点算法的堆叠，而应形成从视频采集、模型推理、规则判断、状态反馈到前端展示的完整闭环。

另一个重要体会是，边缘 AI 项目必须高度重视硬件资源约束。在桌面端或服务器端，可以较容易地采用更复杂的模型和更重的后处理流程；但在嵌入式开发板上，额外的计算开销、频繁的数据拷贝或不合理的线程调度，都可能直接影响系统实时性。因此，本项目在设计过程中采用了跳帧推理、轻量化规则判断和白名单模板比对等较为务实的工程策略。这些方案并不一定是理论上精度最高或复杂度最高的方案，但更加符合当前 RK3588 端侧部署环境下对实时性、稳定性和资源占用的综合要求。

以家庭模式为例，如果一开始就直接引入完整的大规模深度学习人脸识别框架，虽然理论上可以获得更强的识别能力，但也会带来模型适配、依赖管理、运行开销和部署复杂度等问题。在竞赛作品的实现阶段，采用“人脸检测—白名单录入—模板比对—陌生人告警”的轻量化闭环路径，反而更有利于在有限硬件资源和有限开发周期内完成稳定展示，体现了嵌入式项目中工程取舍的重要性。

模块化设计也是本项目研发中的重要收获。系统将模型推理逻辑封装在 `detector.py` 中，将视频采集、跳帧调度和场景规则判断放在 `camera.py` 中，将 Web 路由、接口管理和前端交互放在 `app.py` 中。这样的分层设计看似基础，却显著提升了系统的可维护性和协同开发效率。当需要增加新场景时，主要修改业务规则模块；当需要更换检测模型时，尽量只调整推理引擎层；当前端界面需要

优化时，也不会直接影响底层视频采集与推理主循环。对于竞赛作品而言，清晰的软件架构不仅有助于开发调试，也有助于向评审展示系统的完整性和可扩展性。

最后，本项目使我们更加明确地认识到，优秀的技术展示并不是简单堆砌算法名词或硬件参数，而是要让技术能力通过自然、可信、连贯的系统行为被直观感受到。评审所看到的不应只是抽象的代码和模型，而应是一个打开浏览器即可查看实时画面、切换应用模式、触发告警规则、录入白名单并观察陌生人识别结果变化的完整系统。这个从算法实现走向系统表达的过程，是本项目带给我们的重要收获，也为后续继续开展边缘智能、嵌入式 AI 和多场景安防系统研发积累了宝贵经验。

参考文献

- [1] 房子靖,韩玉兰,李聪,等.基于改进 YOLOv11n 的监控场景暴力行为人员检测算法[J].物联网技术,2026,16(13):18-23+29.[2]商舒雯,田禾,刘洁,等.
- [2] 基于 RSB-YOLO 的课堂学生行为检测算法研究[J/OL].光电子·激光,1-12[2026-07-07]
- [3] 周泉泉,梅健强.RLA-YOLO 遥感目标检测算法[J].天津职业技术师范大学学报,2026,36(2):43-49.
- [4] 程崑君.基于 TensorFlow Lite 的嵌入式设备深度学习推理部署实现[J].中国信息化,2026,(6):52-53.
- [5] 才华,李林,王璐,等.基于国产嵌入式平台的实时目标跟踪算法[J].空间控制技术与应用(中英文),2026,52(3):20-33.
- [6] 曹可兵,张伟,蔡振宇.基于 YOLOv8 的小目标轻量化检测模型研究[J].自动化应用,2026,67(10):28-34+39.
- [7] 杨磊,王慧辰,张燕妮.基于多目标识别的校园人流检测方法[J].科技与创新,2026,(8):61-63.DOI:10.15913/j.cnki.kjycx.2026.08.016.
- [8] 田川川.关于嵌入式系统在家庭智能系统中的应用分析[J].中国高新科技,2025,(18):118-119+122.
- [9] 梁宵,刘宗元,修一伟,等.基于 DiMP 算法的安防巡检系统设计与开发[J].沈阳航空航天大学学报,2025,42(2):63-71.
- [10] 李松涛.人工智能技术在实时嵌入式多核系统中的应用[J].电子元器件与信息技术,2025,9(4):102-104+108.
- [11] 张宝燕.基于图像处理的二阶段人流检测系统研究[J].山西电子技术,2023,(2):105-107+110.
- [12] 连俊芳.基于嵌入式的行人检测、追踪智能安全防控系统设计[J].技术与市场,2022,29(9):19-22.
- [13] 应忍冬,刘佩林.AI 嵌入式系统: 算法优化与实现[M].机械工业出版社:2022.
- [14] 苏秉华,吴红辉,滕悦然.人工智能(AI)应用从入门到精通[M].化学工业出版社:2020.
- [15] 汪康,刘军清,陈鹏.面向人流密度检测的边缘提取算法研究[J].信息通信,2018,(8):36-38.